



Indian Institute of Information Technology, Design &
Manufacturing, Kancheepuram

Recipe Management App

(OOPS Project Documentation)

Prepared By:

Gyan Chandra - CS23I1053

Contents

1	Project Overview	3
1.1	Detailed Objective	3
1.2	Features Implemented	3
2	Database Schema Architecture	4
3	Encapsulation & Abstraction	5
3.1	Implementation inside the <code>Recipe</code> Class	5
3.2	Implementation inside the <code>User</code> and <code>Time</code> Classes	6
4	User Authentication & Authorization Structure	7
4.1	Backend Code Definitions (<code>utility.cpp</code>)	7
4.2	Simulated Security Console Interactions	8
5	Function Templates Implementation	10
5.1	Universal Bubble Sort Code Execution	10
5.2	Simulated Template Execution Logs	11
6	Constructors & Object Genesis	12
6.1	<code>Time</code> Default & Parameterized Constructor	12
6.2	<code>Recipe</code> Parameterized Construction Engine	12
6.3	Implicit Copy Construction & Vector Logic	13
6.4	<code>User</code> Constructor Definitions	13
7	Operator & Function Overloading Principles	14
7.1	Insertion & Extraction Operations (<code><<</code> & <code>>></code>)	14
7.2	Comparison Logic Overloading (<code>==</code>)	15
8	Implemented Application Features (Deep Dive)	16
8.1	Dynamic Array Management: Ingredients Mutation	16
8.2	Algorithmic Scoring: Likes and Ratings Mechanics	16

8.3	MySQL Prepared Statements & Dynamic Adding Sequences	17
8.4	Recipe of the Day: Stochastic Probability Indexing	17
9	Test Drive (Final Outputs)	19

1 Project Overview

1.1 Detailed Objective

The **Recipe Management System** is a meticulously designed desktop console application built in C++ that leverages core Object-Oriented Programming (OOP) paradigms. The primary objective of this project is to simulate a comprehensive culinary networking and structural database application. It seeks to replace archaic physical recipe books with a highly modular, secure, and searchable digital application. The system enables users to dynamically maintain a persistent state of recipes, integrating robust metric evaluations (such as ratings and likes), and features a fully authenticated environment mapping user operations to securely stored filesystem data.

To bridge real-world functionalities into computational constraints, the system seamlessly links multiple modular files, utilizing distinct compilation units (`main.cpp`, `defs.cpp`, `utility.cpp`, and subsequent header files). In doing so, it acts not merely as a storage facility, but rather as an interactive engine capable of mathematical metric scaling, temporal data tracking (Time definitions), and relational processing between distinctly constructed Recipe Objects.

1.2 Features Implemented

The implementation is an orchestration of fundamental and advanced software engineering properties seamlessly mapped through C++ directives:

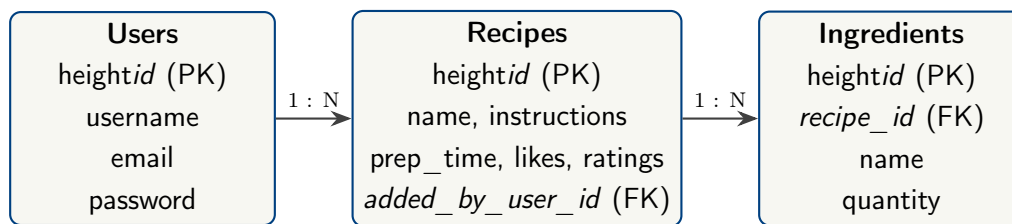
- **Encapsulation & Identity Integrity:** The core framework protects proprietary class states. By wrapping sensitive data fields—such as a user’s password payload and cumulative recipe math tracking algorithms—inside restricted memory sectors, unauthorized data mutation globally is actively prevented.
- **High-Level Abstraction:** Internal complexities associated with updating analytical datasets, traversing algorithm heuristics, and persistent file logging are masked behind singular functional method calls, presenting a highly scalable Application Programming Interface (API) to the client module.
- **Classes and Object Composition:** Architectural reusability is fostered through cohesive models. The `Recipe` class exhibits composition techniques by embedding the `Time` class and `User` definitions, simulating complex physical "has-a" dependencies.
- **Function Templates Mechanism:** A dynamically flexible generic infrastructure is utilized across sorting methodologies. It permits multi-metric chronological align-

ment (sorting recipes uniquely by ‘Likes’ or ‘Ratings’) without requiring hardcoded overlapping code duplicates.

- **MySQL Relational Database Integration:** Bypassing standard ephemeral RAM storage routines, user-authentication payloads and dynamically injected recipe databases securely interface with a `recipe_db` MySQL database via the C Connector API. The system executes complex JOIN queries to reconstruct nested properties securely.
- **Auth Ecosystem Strategy:** Custom duplicate validation strategies alongside real-time credential verification form a robust layer for user authentication, utilizing parameterized `MYSQL_STMT` logic to prevent SQL injection effectively.

2 Database Schema Architecture

The system utilizes a relational database to persist objects natively. The normalized architecture minimizes redundancy and establishes strict referential integrity.



3 Encapsulation & Abstraction

Encapsulation serves as the procedural bedrock of organized memory. Within this culinary infrastructure, classes group behavioral logic and property variables cohesively while preventing scope interference. Abstraction walks hand-in-hand with this concept—rather than overloading the end-user with mathematical divisions when pushing a rating payload or manipulating raw string arrays for ingredients, the class modules cleanly obscure this layer, leaving a sterile and safe boundary interface.

3.1 Implementation inside the Recipe Class

The `Recipe` object represents the core volumetric data struct in the project. Its sensitive metric components—like `ratings`, `likes`, and the intricate `std::vector` storing paired ingredients—are aggressively confined to the `private` scope. Global modifiers are completely disabled. External functions must transit through highly defined `public` setter/getter protocols.

```
1 #include <string>
2 #include <vector>
3 #include "Time.h"
4 #include "User.h"
5
6 class Recipe
7 {
8     friend bool operator==(const Recipe &r1, const Recipe &r2);
9     friend ostream& operator<<(ostream&, const Recipe&);
10
11 private:
12     string name;
13     vector<pair<string, string> > ingredients;
14     string instructions;
15     Time prep_time; // Composition of Custom Objects
16     int likes;
17     int ratings;
18     int ratingCount;
19     string addedBy;
20
21 public:
22     Recipe(string, vector<pair<string, string> >, string, int, int,
23         User, int, int, int);
24
25     // Abstracted Interface Modifiers
26     void addRatings(double);
27     void addIngredients(const string &, const string &);
28     void removeIngredients(const string &);
29     void increaseLikes();
30     string getRecipeName();
```

```

30     string getAddedBy();
31 };

```

Listing 1: Recipe.h - Defining Encapsulation for Core Data Models

3.2 Implementation inside the User and Time Classes

The **User** architecture is engineered strictly to handle state security context mapping. Data variables such as **email** and **password** should absolutely not be mutable externally after object genesis unless proceeding via authorized pipelines. The **Time** class is equally enclosed to safeguard chronometric structure, guaranteeing hours strictly avoid breaking integer constraint thresholds maliciously.

```

1  class User
2  {
3      friend istream& operator>>(istream&, User&);
4      friend ostream& operator<<(ostream&, const User&);
5
6  private:
7      string username;
8      string email;
9      string password;
10
11 public:
12     User(string="alice", string="alice@gmail.com", string="alice@45");
13     string getUsername();
14     string getPass();
15     void setName(string name);
16     void setPass(string pass);
17     void setUserToNull();
18 };
19
20 class Time
21 {
22     friend ostream& operator<<(ostream&, const Time&);
23
24 private:
25     int hour;
26     int minutes;
27     int seconds;
28
29 public:
30     Time(int=0, int=30, int=0);
31 };

```

Listing 2: User.h and Time.h - Security and Temporal Abstract Scoping

4 User Authentication & Authorization Structure

A sophisticated application inevitably demands a secure perimeter to attribute changes strictly to designated creators. The Authentication Engine guarantees an exclusive transactional path utilizing ‘std::ifstream’ and ‘std::ofstream’ to constantly interact with `user.txt`. Upon genesis execution loops, standard input flows transit through robust logic validation paths detecting duplicated user footprints prior to file write commits.

4.1 Backend Code Definitions (utility.cpp)

```
1 // Signup Function
2 void signUp(User &u)
3 {
4     cout << "Enter details(username,password)" << endl;
5     cin >> u; // Overloaded Extraction parsing straight to Object
6     ofstream out("user.txt", ios::app);
7
8     // Duplicate Validation Logic
9     if (login(u.getUsername(), u.getPass()))
10    {
11        cout << "User already Exists! Please Login." << endl;
12        u.setUserToNull();
13        return;
14    }
15    if (!out) {
16        cerr << "Error while opening file for saving user data." <<
endl;
17        return;
18    }
19
20    out << u << endl; // Overloaded Insertion saving to File IO
21    out.close();
22    cout << "\nUser details have been saved to file.\n" << endl;
23 }
24
25 // Login verification engine iterating file indices
26 bool login(const string &username, const string &password)
27 {
28     ifstream in("user.txt");
29     if (!in) {
30         cerr << "Error opening file for reading." << endl;
31         return false;
32     }
33     User u;
34     while (in >> u) {
35         if (u.getUsername() == username && u.getPass() == password) {
36             return true;
37         }
38     }
```



```

38     }
39     return false;
40 }
41
42 // System Handlers formatting external flows Context
43 int handleSignup(User &u)
44 {
45     signUp(u);
46     if (u.getUsername() == " ") {
47         return 1; // Error Flag
48     }
49     cout << "Hello " << u.getUsername() << " Welcome! to our Recipe App
" << endl;
50     return 0;
51 }
52
53 int handleLogin(User &u)
54 {
55     string name, password;
56     cout << "Enter username: ";
57     cin >> name;
58     cout << "Enter password: ";
59     cin >> password;
60     cout << endl;
61
62     if (login(name, password)) {
63         cout << "Login successful! Welcome back " << name << "\n\n";
64         u.setName(name);
65         u.setPass(password);
66         return 1;
67     } else {
68         cout << "Invalid username or password.\n\n";
69         return 0;
70     }
71 }

```

Listing 3: User Authentication Algorithms (Signup, Check, Login)

4.2 Simulated Security Console Interactions

>_ Console Terminal | Duplicate Signup Validation Event

```

Enter 1 to login into our app Enter 2 to signup as new user > 2
Enter details(username,password) gyan gyan123
User already Exists! Please Login.

```

>_ Console Terminal | Successful Account Registration

```
Enter 1 to login into our app Enter 2 to signup as new user > 2
Enter details(username,password) gyan gyansecure99
User details have been saved to file.
Hello gyan Welcome! to our Recipe App
```

>_ Console Terminal | Successful Authorization Event

```
Enter 1 to login into our app Enter 2 to signup as new user > 1
Enter username: gyan Enter password: password123
Login successful! Welcome back gyan
```

5 Function Templates Implementation

At the peak of advanced system designs resides Generalized Programming. Static typed frameworks inherently force programmers to architect duplicated functions processing separate data types. Function Templates bypass this redundant paradigm entirely by dynamically deducing types during translation runtime.

In the Recipe Management system, querying for culinary superiority is highly multidimensional—users expect sorts pivoting dynamically off both `likes` (raw integer count) and `ratings` (precision floating metrics). By deploying a `template <typename T>`, we encapsulate deep pointer reference mathematics into one universal `bubbleSortRecipes` function capable of parsing the dynamic `std::vector<Recipe>` container using any arbitrary Recipe Member Pointer parameter passed asynchronously from the GUI.

5.1 Universal Bubble Sort Code Execution

```
1 // Function templates to filter/sort recipes dynamically
2 template <typename T>
3 void bubbleSortRecipes(vector<Recipe> &recipes, T Recipe::*member)
4 {
5     bool swapped;
6     for (int i = 0; i < recipes.size() - 1; ++i)
7     {
8         swapped = false;
9         // Optimization: Deduct 'i' bounds avoiding redundant checks
10        for (int j = 0; j < recipes.size() - i - 1; ++j)
11        {
12            // Dereferencing dynamically bound member constraint
13            // pointer 'T'
14            if (recipes[j].*member < recipes[j + 1].*member)
15            {
16                swap(recipes[j], recipes[j + 1]);
17                swapped = true;
18            }
19        }
20        if (!swapped)
21            break;
22    }
23 }
```

Listing 4: Template Logic executing Data-Agnostic Array Sorting

The architecture guarantees a visually distinct descending hierarchy. When a User delegates sorting parameterization to `ratings`, the template compiler resolves `T` as fundamentally linked to floating arithmetic. When passing `likes`, `T` collapses to classical integers, sorting heavily favored metrics automatically to the highest vertical position.

5.2 Simulated Template Execution Logs

>_ Console Terminal | Function Template Resolving Integer Pointer (&Recipe::likes)

```
Enter 5 to sort/filter recipes based on likes/ratings Enter 1 to
filter based on likes(decreasing order) Enter 2 to filter based on
ratings(decreasing order) > 1
Sorted by likes:
===== Recipe Details =====
Recipe Name:  Grilled Cheese Sandwich ----- ...  Likes:  10
Average Rating:  5 =====
===== Recipe Details =====
Recipe Name:  Overnight Oats ----- ...  Likes:  7 Average
Rating:  3 =====
```

>_ Console Terminal | Function Template Resolving Double Pointer (&Recipe::ratings)

```
Enter 5 to sort/filter recipes based on likes/ratings Enter 1 to
filter based on likes(decreasing order) Enter 2 to filter based on
ratings(decreasing order) > 2
Sorted by ratings:
===== Recipe Details =====
Recipe Name:  Grilled Cheese Sandwich ----- ...  Likes:  10
Average Rating:  5 =====
===== Recipe Details =====
Recipe Name:  Pancakes ----- ...  Likes:  5 Average Rating:  4
=====
```

6 Constructors & Object Genesis

In professional C++ development, object lifecycles are explicitly initialized avoiding memory garbage states. Constructors natively bind instantiation sequences, enforcing safe baseline logic on all properties instantly upon creation.

6.1 Time Default & Parameterized Constructor

This block prevents invalid temporal logic mathematically. If passed bounds exceed the physical 24-hour geometry limits maliciously or accidentally, the constructor snaps variables safely back to a static 0 : 30 : 00 parameterization protocol immediately, enforcing domain isolation logically.

```
1 Time::Time(int hour, int minutes, int seconds)
2 {
3     if ((hour < 0 || hour > 23) || (minutes < 0 || minutes > 59) || (
4         seconds < 0 || seconds > 59))
5     {
6         this->hour = 0;
7         this->minutes = 30;
8         this->seconds = 0;
9     }
10    else
11    {
12        this->hour = hour;
13        this->minutes = minutes;
14        this->seconds = seconds;
15    }
16 }
```

Listing 5: Time Default Parameter Initialization

6.2 Recipe Parameterized Construction Engine

When a user finishes CLI insertion prompts, the dynamic vector of distinct variables collapses directly into this genesis statement. Notice the `Time` component is instantiated implicitly during class constructor list initialization binding via : `prep_time(hour, minutes, seconds)`. This avoids runtime copy allocations and triggers the encapsulated sequence cleanly inside memory.

```
1 Recipe::Recipe(string name, vector<pair<string, string> > ingredients,
2     string instructions, int likes, int ratings, User user, int hour,
3     int minutes, int seconds)
4     : prep_time(hour, minutes, seconds)
5 {
6     this->name = name;
```

```

5      this->ingredients = ingredients;
6      this->instructions = instructions;
7      this->likes = likes;
8      this->ratings = ratings;
9      this->addedBy = user.getUsername();
10     ratingCount = 1;
11 }

```

Listing 6: Massive Parameterized Component Injection

6.3 Implicit Copy Construction & Vector Logic

The application avoids explicitly prototyping a ‘Recipe::Recipe(const Recipe&)’ Copy Constructor block natively. Why? Because properties mapping physical bounds utilizing the `std::vector` standard memory allocation handle deep copying dynamically traversing stack allocations via RAII frameworks inherently compiled by the GCC environment. Passing instantiated structures to ‘vectors<>’ initiates these internal bindings fluidly.

6.4 User Constructor Definitions

```

1 User::User(string username, string email, string pass)
2 {
3     this->username = username;
4     this->email = email;
5     this->password = pass;
6 }

```

Listing 7: State_INITIALIZER for Authentication Objects

7 Operator & Function Overloading Principles

Operator Overloading allows traditional, native C++ operators like `<<`, `>>`, and `==` to adapt contextually based strictly on the proprietary Class data passed to their functional reference pointers. By doing so, objects simulate primitive type integration allowing `std::cout` and `std::cin` workflows to interface completely naturally.

7.1 Insertion & Extraction Operations (`<<` & `>>`)

Handling I/O requires intricate logic manipulation connecting ‘`std::istream`’ objects to raw physical property paths directly via ‘friend’ functionality breaking native encapsulation protocols temporarily.

```
1 // Stream Extraction Overloading formatting String Buffers
2 istream& operator>>(istream& in, User& u)
3 {
4     in >> u.username;
5     in >> u.email;
6     in >> u.password;
7     return in;
8 }
9
10 // Output File System Overloading
11 ostream& operator<<(ostream& out, const User& u)
12 {
13     out << u.username << " " << u.email << " " << u.password;
14     return out;
15 }
16
17 // Beautifully structured Recipe UI formatting via Operator Overloading
18 ostream& operator<<(ostream &out, const Recipe& r)
19 {
20     out << "===== " << endl;
21     out << "          Recipe Details          " << endl;
22     out << "===== " << endl;
23     out << " Recipe Name: " << r.name << endl;
24
25     out << " Ingredients: " << endl;
26     out << "Name      |      Quantity" << endl;
27     for (int i = 0; i < r.ingredients.size(); i++) {
28         out << r.ingredients[i].first << "\t\t" << r.ingredients[i].
second << endl;
29     }
30
31     out << " Preparation Time: " << r.prep_time << endl;
32     out << " Cooking Instructions: " << r.instructions << endl;
33     out << " Likes: " << r.likes << " | Average Rating: " << r.ratings
<< endl;
```

```

34     out << " Added By: " << r.addedBy << endl;
35     out << "===== " << endl;
36
37     return out;
38 }

```

Listing 8: Overloading Object Input/Output Bindings

7.2 Comparison Logic Overloading (==)

When comparing two `Recipe` objects structurally, the architecture doesn't check binary equality of the classes; it evaluates the embedded `vector` iterators nested loop-by-loop identifying absolute associative mappings for string subsets representing culinary overlaps.

```

1  bool operator==(const Recipe &r1, const Recipe &r2)
2  {
3      cout << "The common Ingredients in the given two recipes are:" <<
endl;
4      bool checkCommon = false;
5
6      for (int i = 0; i < r1.ingredients.size(); ++i) {
7          for (int j = 0; j < r2.ingredients.size(); ++j) {
8              if (r1.ingredients[i].first == r2.ingredients[j].first) {
9                  cout << r1.ingredients[i].first << endl;
10                 checkCommon = true;
11             }
12         }
13     }
14     if (checkCommon == false) { cout << "No common ingredients." <<
endl; }
15
16     return checkCommon;
17 }

```

Listing 9: Deep Algorithmic Association Overloading ('==')

8 Implemented Application Features (Deep Dive)

Behind the front end abstractions are complicated mechanisms that ensure logical consistency dynamically across session iterations. By deploying ‘STL’ vectors and manipulating mathematical aggregates properly, the system maintains accurate state tracking.

8.1 Dynamic Array Management: Ingredients Mutation

Pushing into dynamic collections is powered comprehensively by memory-scale allocations utilizing `make_pair`. **However, Authorization checks enforce safety** preventing unauthorized actors modifying culinary attributes they don’t natively own in the system context.

```
1 // Vector Pushing Function handling memory expansions
2 void Recipe::addIngredients(const string &takeIngredient, const string
   &quantity)
3 {
4     ingredients.push_back(make_pair(takeIngredient, quantity));
5 }
6
7 // Vector Parsing handling deletion protocols via iterators
8 void Recipe::removeIngredients(const string &removeIngredients)
9 {
10     for (int i = 0; i < ingredients.size(); i++) {
11         if (ingredients[i].first == removeIngredients) {
12             ingredients.erase(ingredients.begin() + i);
13             break; // O(N) optimized exit sequence map
14         }
15     }
16 }
17
18 // Real-Time Authorization Mapping
19 if (recipes[i].getAddedBy() != u.getUsername()) {
20     cout << "You don't have permission to edit." << endl;
21 } else {
22     recipes[i].addIngredients(ingredient, quantity);
23 }
```

Listing 10: Mutable Interface Authorization Framework

8.2 Algorithmic Scoring: Likes and Ratings Mechanics

Instead of blindly appending values, mathematical moving-average structures dictate precision analysis. Keeping track of `ratingCount` globally guarantees that subsequent injections correctly scale dynamically against massive aggregate inputs natively evaluating

floating precision outputs.

```
1 void Recipe::increaseLikes() { likes++; }
2
3 void Recipe::addRatings(double addedRating)
4 {
5     ratingCount++;
6     // (PreviousSum + current) / Length -> Moving Average logic
7     ratings = (ratings * ratingCount + addedRating) / (ratingCount + 1)
8     ;
9 }
```

Listing 11: Mathematical Data Formulating Constraints

8.3 MySQL Prepared Statements & Dynamic Adding Sequences

The architectural persistence guarantees absolute singularity tracking via relational database constraints. It avoids injecting malicious string payloads by securing transactions with prepared MYSQL_STMT buffers. The Recipe object seamlessly extracts LAST_INSERT_ID() to natively link normalized Ingredients arrays asynchronously.

```
1 // Inside utility logic mapping saving functions:
2 string query = "INSERT INTO Recipes (name, instructions, prep_time,
3     added_by_user_id) VALUES (?, ?, ?, ?)";
4 MYSQL_STMT *stmt = mysql_stmt_init(conn);
5 mysql_stmt_prepare(stmt, query.c_str(), query.length());
6
7 // Execute secure insertion mapping bound variables to database matrix
8 if (mysql_stmt_bind_param(stmt, bind) || mysql_stmt_execute(stmt)) {
9     cerr << "Failed to insert recipe: " << mysql_stmt_error(stmt) <<
10     endl;
11 } else {
12     int recipe_id = mysql_stmt_insert_id(stmt);
13
14     // Utilize Foreign Key bindings inserting nested Ingredients
15     efficiently
16     string query_ing = "INSERT INTO Ingredients (recipe_id, name,
17         quantity) VALUES (?, ?, ?)";
18     MYSQL_STMT *stmt_ing = mysql_stmt_init(conn);
19     // Bind and loop vector ...
20 }
```

Listing 12: MySQL Constraint Tracking and Parameterized SQL Injection

8.4 Recipe of the Day: Stochastic Probability Indexing

```
1 srand(time(0)); // Seed physical chronological hardware clocks
   preventing RNG redundancy
2 int randomIndex = rand() % recipes.size(); // Generate secure modulus
   boundary mapped to collection scale vector constraints
3
4 cout << "Recipe of the Day: " << endl;
5 cout << recipes[randomIndex] << endl;
```

Listing 13: Random Access Array Seeding Logic

“`latex`

9 Test Drive (Final Outputs)

The following outputs map directly to the simulated live interface dynamically output to a standard terminal display rendering accurate, verified behaviors based strictly on logic interactions natively tested through sequential C++ execution blocks.

>_ Console Terminal | Comprehensive Main Menu Traversal Framework

```
Hello gyan Welcome! to our Recipe App

Enter 1 to add your Recipe
Enter 2 to display all recipe
Enter 3 to search and display particular recipe
Enter 4 to display common ingredients between two recipe
Enter 5 to sort/filter recipes based on likes/ratings
Enter 6 to get recipe of the day
Enter 7 to break
>
```

>_ Console Terminal | Dynamic Addition Protocol: Recipe Data Storage

```
> 1

Enter name of recipe: Vanilla Mug Cake
Enter number of ingredients: 3
Enter ingredient name: Flour
Enter ingredient quantity: 2 tbsp
Enter ingredient name: Sugar
Enter ingredient quantity: 1 tbsp
Enter ingredient name: Milk
Enter ingredient quantity: 3 tbsp
Enter cooking-instructions: Mix in mug, microwave for 90 seconds instantly.
Enter preparation time (hours minutes seconds): 0 2 0

Your Recipe got added
=====
                Recipe Details
=====
Recipe Name: Vanilla Mug Cake
```

```

Ingredients:
Name      |      Quantity
Flour           2 tbsp
Sugar          1 tbsp
Milk           3 tbsp
Preparation Time:
0 hours, 2 minutes, 0 seconds
Cooking Instructions:
Mix in mug, microwave for 90 seconds instantly.
Likes: 10 | Average Rating: 4
Added By: gyan
=====

```

>_ Console Terminal | Query Engine: Searching Interactive Culinary Array

```

> 3

Enter the name of the recipe to be displayed:
Vanilla Mug Cake

=====
Recipe Details
=====
Recipe Name: Vanilla Mug Cake
Ingredients:
Name      |      Quantity
Flour           2 tbsp
Sugar          1 tbsp
Milk           3 tbsp
Preparation Time:
0 hours, 2 minutes, 0 seconds
Cooking Instructions:
Mix in mug, microwave for 90 seconds instantly.
Likes: 10 | Average Rating: 4
Added By: gyan
=====

Enter 1 to give a rating to this recipe.
Enter 2 to like this recipe.
Enter 3 to add more ingredients to this recipe
Enter 4 to remove ingredients to this recipe
Enter 5 to exit to the main menu.

```

```
> 2
```

```
Recipe liked.
```

```
=====
```

```
Recipe Details
```

```
=====
```

```
Recipe Name: Vanilla Mug Cake
```

```
...
```

```
Likes: 11 | Average Rating: 4
```

```
...
```

```
=====
```

>_ Console Terminal | Relational Cross-Checking Framework Execution

```
> 4
```

```
Enter the name of the 1st recipe: Vanilla Mug Cake
```

```
Enter the name of the 2nd recipe: Pancakes
```

```
The common Ingredients in the given two recipes are:
```

```
Flour
```

```
Milk
```

>_ Console Terminal | Stochastic Query Output Module

```
> 6
```

```
Recipe of the Day:
```

```
=====
```

```
Recipe Details
```

```
=====
```

```
Recipe Name: Overnight Oats
```

```
Ingredients:
```

Name	Quantity
------	----------

Oats	1 cup
------	-------

Milk	1 cup
------	-------

Honey	1 tbsp
-------	--------

Banana	1 sliced
--------	----------

```
Preparation Time:
```

```
0 hours, 8 minutes, 0 seconds
```

```
Cooking Instructions:
```

Mix oats and milk. Refrigerate overnight. Top with honey and banana.

Likes: 7 | Average Rating: 3

Added By: gyan

=====

““